

Introducción a los Sistemas Operativos

Entrada / Salida - II



- ✓ Versión: Septiembre 2012
- ✓ Palabras Claves: Entrada , Salida, Dispositivos, Interrupciones, DMA, Drivers, Windows, Linux

Algunas diapositivas han sido extraídas de las ofrecidas para docentes desde el libro de Stallings (Sistemas Operativos) y el de Silberschatz (Operating Systems Concepts). También se incluyen diapositivas cedidas por Microsoft S.A.



Diseño del SO - Metas

☑ Eficiencia

- ✓ Los dispositivos de I/O pueden resultar extremadamente lentos respecto a la memoria
- ✓ El uso de la multiprogramación permite que los procesos esperen por la finalización de la I/O mientras que otro se ejecuta
- ✓ I/O no puede alcanzar la velocidad de la CPU



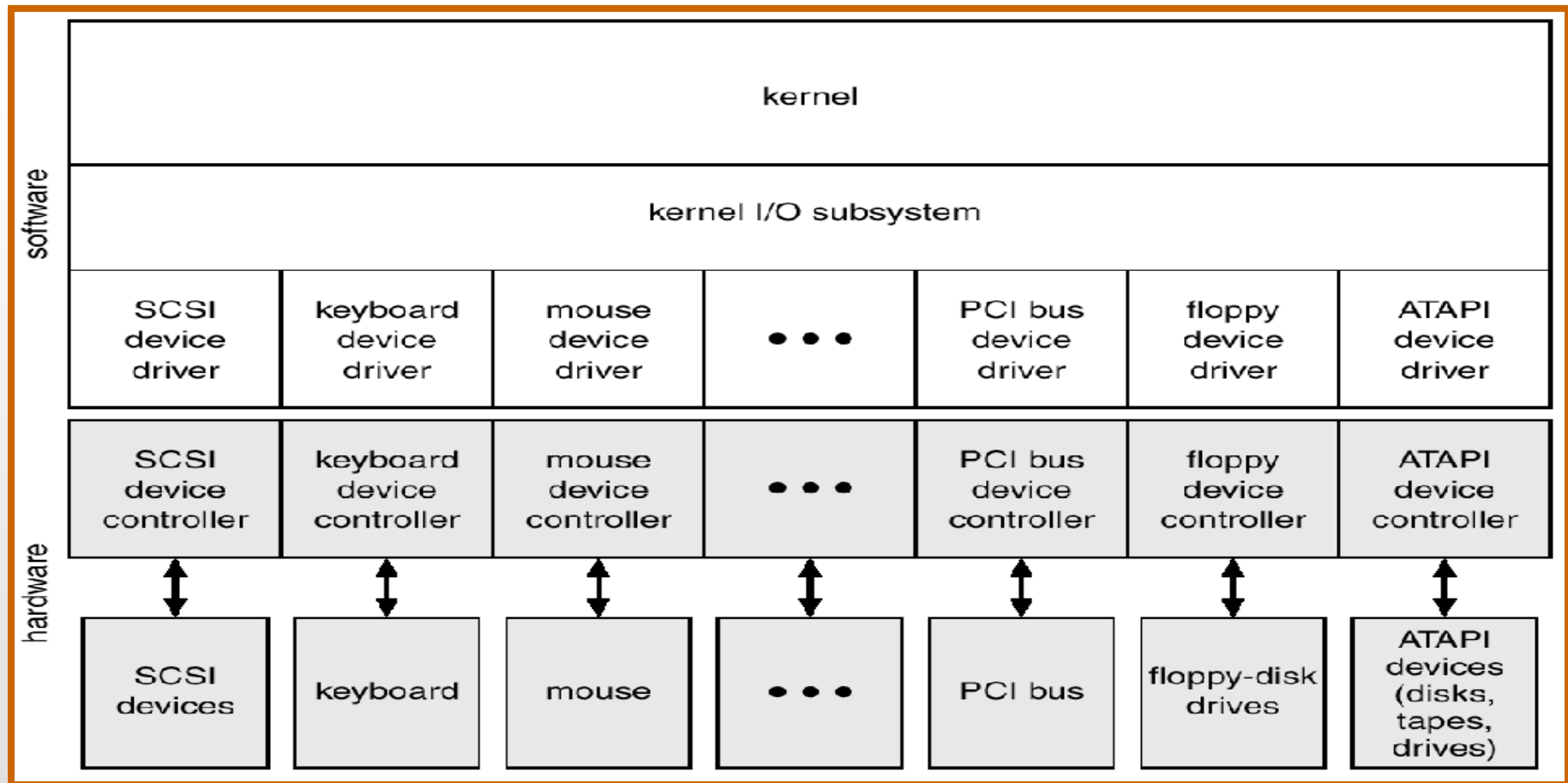
Diseño del SO - Metas (cont.)

☑ Generalización

- ✓ Es deseable manejar todos los dispositivos de I/O de una manera uniforme
- ✓ Ocultar la mayoría de los detalles del dispositivo en las rutinas de niveles más “bajos” para que los procesos vean a los dispositivos, en términos de operaciones comunes como: read, write, open, close, lock, unlock



Subsistema de I/O



Subsistema de I/O - Servicios

✓ Planificación

- ✓ Organización de los requerimientos a los dispositivos
- ✓ Ej: Planificación de requerimientos a disco para minimizar movimientos

✓ Buffering - Almacenamiento de los datos en memoria mientras se transfieren

- ✓ Solucionar problemas de velocidad entre los dispositivos
- ✓ Solucionar problemas de tamaño y/o forma de los datos entre los dispositivos



Subsistema de I/O - Servicios

(cont.)

- ☑ Caching – Mantener en memoria copia de los datos de reciente acceso para mejorar performance
- ☑ Spooling – Mantener la salida a un dispositivo
 - ✓ Si el dispositivo no puede atender el requerimiento en el momento
 - ✓ Ejemplo: Impresora
- ☑ Reserva de Dispositivos: Acceso exclusivo



Subsistema de I/O - Servicios

(cont.)

☑ Manejo de Errores:

- ✓ El S.O. debe administrar errores ocurridos (lectura de un disco, dispositivo no disponible, errores de escritura)
- ✓ La mayoría retorna un número de error o código cuando la I/O falla.
- ✓ Logs de errores



Subsistema de I/O - Servicios

(cont.)

☑ Formas de realizar I/O

- ✓ Bloqueante: El proceso se suspende hasta que el requerimiento de I/O se completa
 - ♦ Fácil de usar y entender
 - ♦ No es suficiente bajo algunas necesidades
- ✓ No Bloqueante: El requerimiento de I/O retorna en cuanto es posible
 - ♦ Se implementa usando multi-threading
 - ♦ Ejemplo: interfase de usuario que recibe input desde el teclado/mouse y se muestra en el screen. Aplicaciones de video que lee frames desde un archivo en disco, y va descomprimiendo y mostrandolo en el screen.



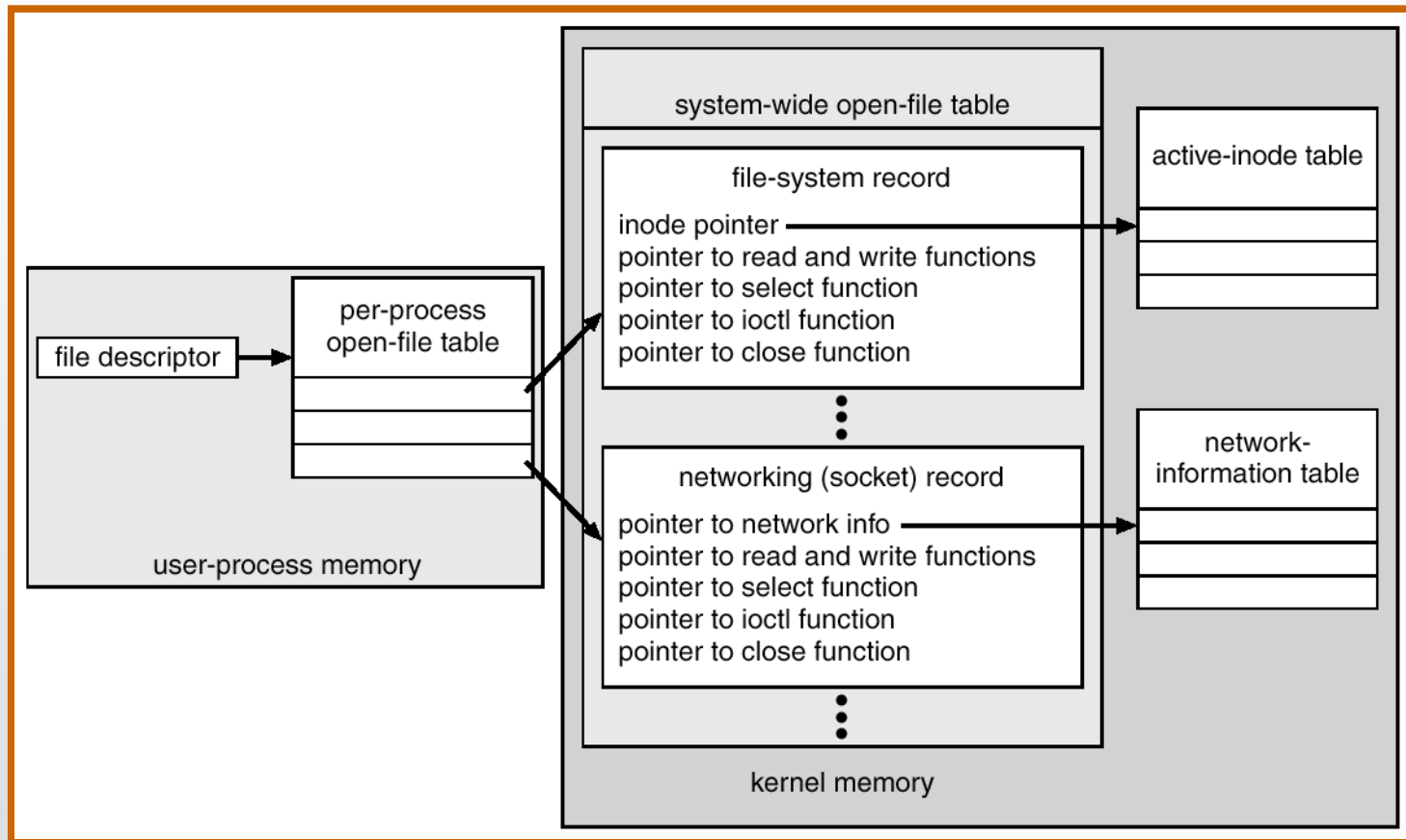
Subsistema de I/O - Estructuras de Datos

- ☑ El Kernel mantiene la información de estado de cada dispositivo o componente
 - ✓ Archivos abiertos
 - ✓ Conexiones de red
 - ✓ Etc.
- ☑ Hay varias estructuras complejas que representan buffers, utilización de la memoria, disco, etc.



Subsistema de I/O - Estructura de Datos

UNIX I/O Kernel Structure

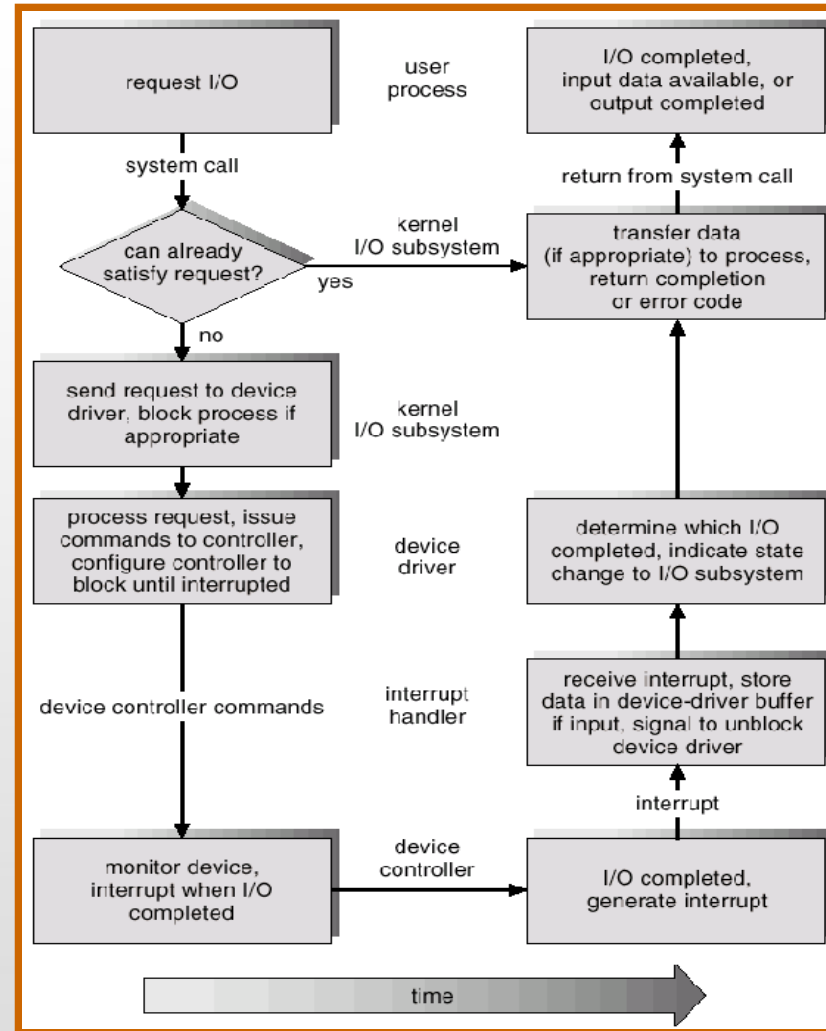


Desde el Requerimiento de I/O hasta el Hardware

- ☑ Consideremos la lectura sobre un archivo en un disco:
 - ✓ Determinar el dispositivo que almacena los datos
 - ✓ Traducir el nombre del archivo en la representación del dispositivo.
 - ✓ Lectura física de los datos en un buffer de memoria
 - ✓ Marcar los datos como disponibles al proceso que realizo el requerimiento
 - ✓ Retornar el control al proceso



Ciclo de vida de un requerimiento de I/O



Subsistema de I/O - Drivers

- ✓ Contienen el código dependiente del dispositivo
- ✓ Manejan un tipo dispositivo
- ✓ Traducen los requerimientos abstractos en los comandos para el dispositivo
 - ✓ Escribe sobre los registros del controlador
 - ✓ Acceso a la memoria mapeada
 - ✓ Encola requerimientos
- ✓ Comúnmente las interrupciones de los dispositivos están asociadas a una función del driver



Subsistema de I/O - Drivers

- ✓ Interfaz entre el SO y el HARD
- ✓ Forman parte del espacio de memoria del Kernel
 - ✓ En general se cargan como módulos
- ✓ Los fabricantes de HW implementan el driver en función de una API especificada por el SO
 - ✓ `open()`, `close()`, `read()`, `write()`, etc
- ✓ Para agregar nuevo HW sólo basta indicar el driver correspondiente sin necesidad de cambios en el Kernel

Driver - Ejemplo en Linux

- ☑ Linux distingue 3 tipos de dispositivos
 - ✓ Carácter: I/O programa o por interrupciones
 - ✓ Bloque: DMA
 - ✓ Red: Ports de comunicaciones
- ☑ Los Drivers se implementan como módulos
 - ✓ Se cargan dinámicamente
- ☑ Debe tener al menos estas operaciones:
 - ✓ `init_module`: Para instalarlo
 - ✓ `cleanup_module`: Para desinstalarlo.



Driver - Ejemplo en Linux (cont.)

- ☑ Operaciones que debe contener para I/O
 - ✓ **open**: abre el dispositivo
 - ✓ **release**: cerrar el dispositivo
 - ✓ **read**: leer bytes del dispositivo
 - ✓ **write**: escribir bytes en el dispositivo
 - ✓ **ioctl**: orden de control sobre el dispositivo



Driver - Ejemplo en Linux (cont.)

- ☑ Otras operaciones menos comunes
 - ✓ `llseek`: posicionar el puntero de lectura/escritura
 - ✓ `flush`: volcar los búferes al dispositivo
 - ✓ `poll`: preguntar si se puede leer o escribir
 - ✓ `mmap`: mapear el dispositivo en memoria
 - ✓ `fsync`: sincronizar el dispositivo
 - ✓ `fasync`: notificación de operación asíncrona
 - ✓ `lock`: reservar el dispositivo
 - ✓



Driver - Ejemplo en Linux (cont.)

- ✓ Por convención, los nombres de las operaciones comienzan con el nombre del dispositivo
- ✓ Por ejemplo, para `/dev/ptr`

```
int ptr_open (struct inode *inode, struct file *filp)

void ptr_release (struct inode *inode, struct file *filp)

ssize_t ptr_read (struct file *filp, char *buff,
                  size_t count, loff_t *offp)

ssize_t ptr_write (struct file *filp, const char *buff,
                  size_t count, loff_t *offp)

int ptr_ioctl (struct inode *inode, struct file *filp,
               unsigned int cmd, unsigned long arg)
```



Driver - Ejemplo en Linux (cont.)

☑ Acceso al hardware

- ✓ Funciones para acceso a los puertos de I/O
<asm/io.h>

```
unsigned inb (unsigned port);  
void outb (unsigned char byte, unsigned port);
```

☑ Leen o Escriben un byte en el puerto indicado

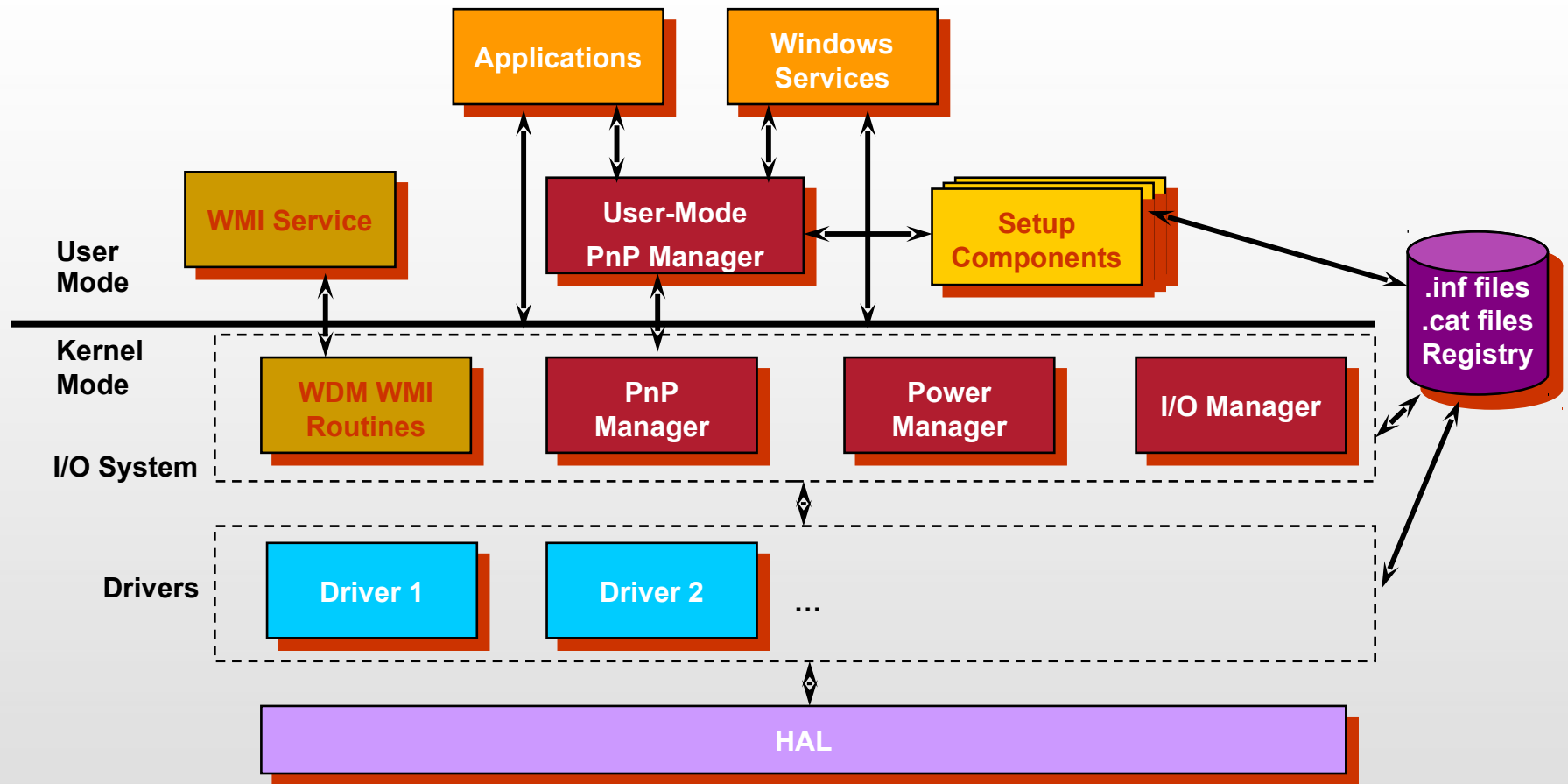


Ejemplo - I/O Windows - Metas

- ✓ Rápida I/O
- ✓ Protección sobre recursos compartidos
- ✓ Proveer servicios comunes a los Drivers
 - ✓ Facilitar el desarrollo de Drivers
 - ✓ Permitir que sean desarrollados en lenguajes de alto nivel.
- ✓ Agregar y remover dinámicamente dispositivos
- ✓ Soporte de múltiples FileSystems (FAT, CDFS, UDF, NTFS)
- ✓ Mapeo de Archivos



Ejemplo -I/O Windows - Componentes



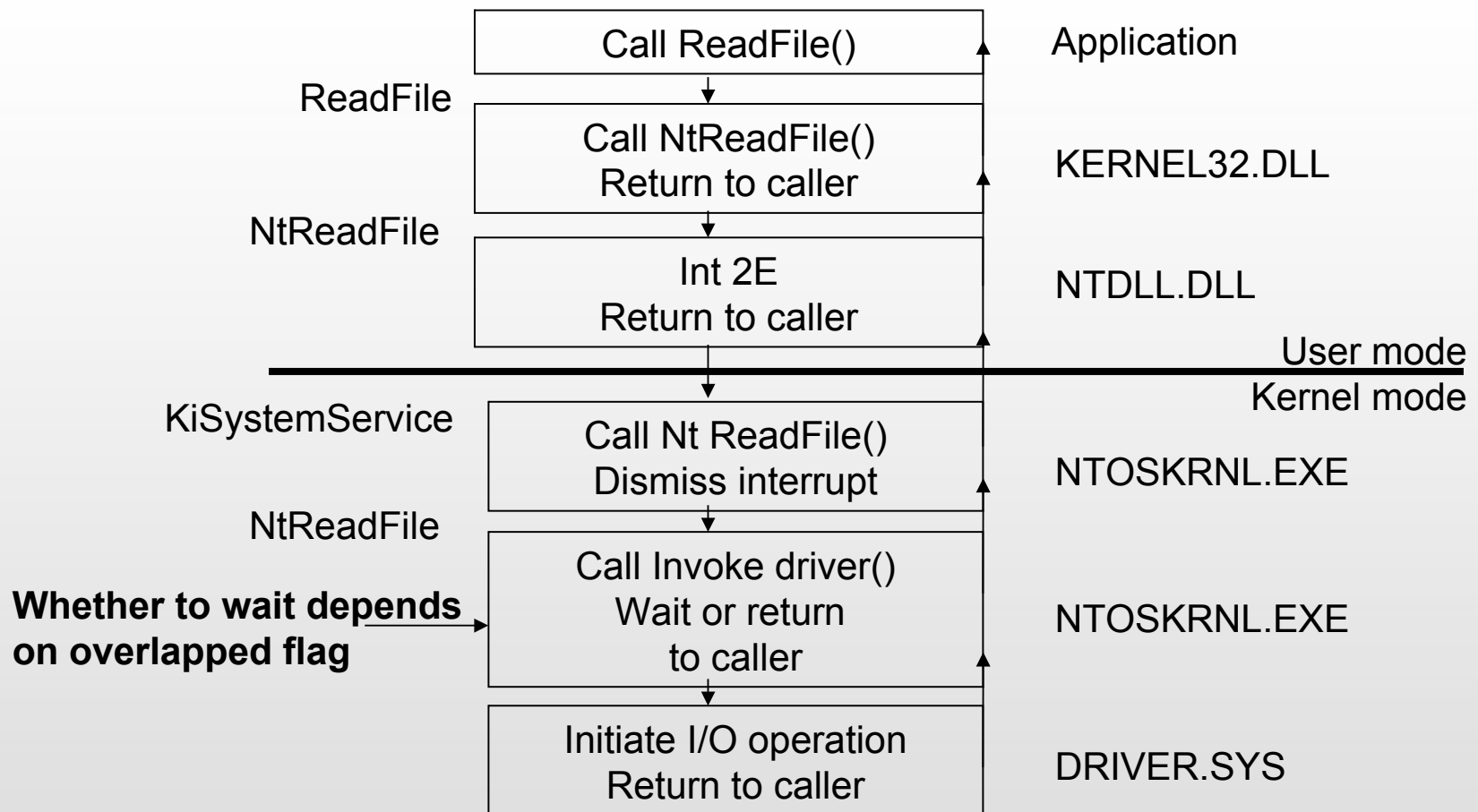
Ejemplo - I/O Windows - Componentes (cont.)

☑ I/O manager

- ✓ Conecta aplicaciones y sistema con dispositivos
- ✓ Contiene una API: ReadFile, WriteFile, CreateFile, CloseFile, DeviceIoControl
- ✓ Contiene la infraestructura que soporta los Drivers
- ✓ Crea los requerimientos (IRP – I/O Request Packet)
- ✓ Envía los IRP al Driver
- ✓ El Driver recibe el IRP y ejecuta la operación
- ✓ El Driver devuelve el IRP al I/O manager



Ejemplo - I/O Windows - Flujo de una operación de I/O



Ejemplo - I/O Windows - HAL

- ✓ Hardware Abstraction Layer
- ✓ Aísla a los Drivers de los aspectos específicos del procesador y HW en general.
- ✓ Provee una API que oculta las diferencias entre las plataformas.
 - ✓ Esencialmente es un “driver” que actúa como bus en el motherboard de la computadora.
 - ✓ Utilizándola, los Drivers son compatibles sobre diferentes arquitecturas.

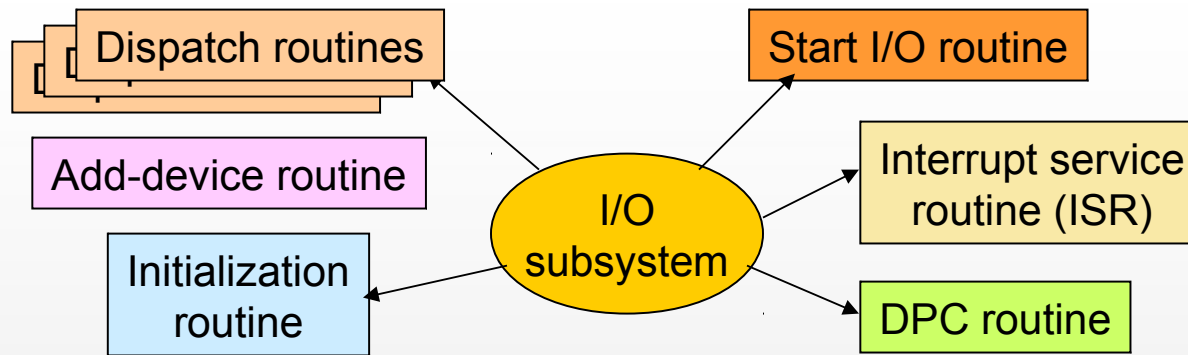


Ejemplo - I/O Windows - WDM - Windows Driver Model

- ✓ Unifica la estructura de los Drivers
- ✓ Para cada tipo de dispositivo define las operaciones que deben implementarse en el Driver
- ✓ Define cómo se pueden comunicar los Drivers
- ✓ Define tipos de drivers
 - ✓ Bus: Manejan un bus. Ej: PCMCIA, PCI, etc.
 - ✓ Function: Manejan un tipo particular de dispositivo. Sound Card, Modem, Discos , etc.
 - ✓ Otros tipos



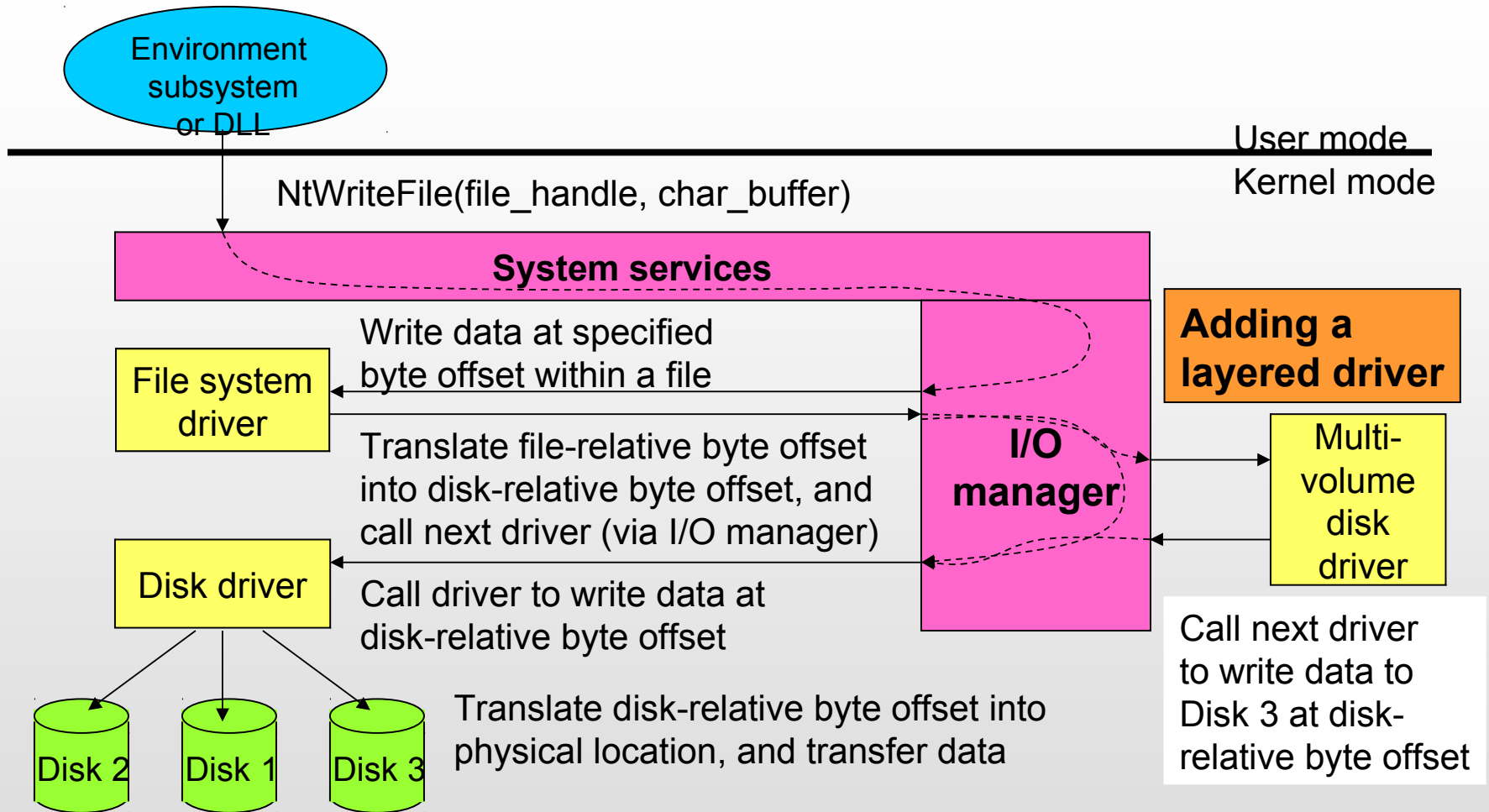
Ejemplo - I/O Windows - Estructura interna de un Driver



- ✓ I/O Manager ejecuta la rutina de inicialización cuando el driver se carga
- ✓ Se ejecuta la rutina add-device cuando se detecta el dispositivo
- ✓ Dispatch Routines: open(), close(), read(), write()
- ✓ Start I/O routine inicializan la transferencia desde/hacia el dispositivo
- ✓ ISR se ejecutan en respuesta a una interrupción



Ejemplo - I/O Windows - Ejemplo - Escribir un archivo



☑ Otros componentes

- ✓ Cancel I/O routine
- ✓ Unload routine
 - ◆ Libera recursos del sistema
- ✓ System shutdown notification routine

☑ Error-logging routines

- ✓ Notifican al I/O manager para generar un registro de error en el archivo de logs.
 - ◆ Ej. bad disk block



Performance

- ☑ I/O es el mayor factor en la performance del sistema:
 - ✓ Utiliza CPU para ejecutar los drivers y el código del subsistema de I/O
 - ✓ Context switches ante las interrupciones
 - ✓ Copia de datos



Mejorar la Performance

- ✓ Reducir el número de context switches
- ✓ Reducir la copia de datos
- ✓ Reducir las interrupciones, utilizando transferencias de gran cantidad de datos,
- ✓ Utilizar DMA

